

ASSIGNMENT-12.5

HTNO:2303A51323

BTNO:19

Prompt:Generate a complete Python program that implements the Merge Sort algorithm.Requirements:

1. Create a function `merge_sort(arr)` that sorts a list in ascending order.
2. Include a proper docstring explaining the algorithm, time complexity, and space complexity.
3. Verify the generated code with test cases.
4. Print the output exactly in the following format:
Original List 1: [elements of the list]
Sorted List 1: [sorted elements]"

```
def merge_sort(arr):  
    """  
    Sorts a list in ascending order using Merge Sort.  
  
    Time Complexity:  
    - Best, Average, Worst Case:  $O(n \log n)$   
  
    Space Complexity:  
    -  $O(n)$   
    """  
    if len(arr) <= 1:  
        return arr  
  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return merge(left, right)  
  
def merge(left, right):  
    result = []  
    i = j = 0  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            result.append(left[i])  
            i += 1  
        else:  
            result.append(right[j])  
            j += 1  
    result.extend(left[i:])  
    result.extend(right[j:])  
    return result
```

```

    return result

# ----- Test Cases -----
test_lists = [
    [38, 27, 43, 3, 9, 82, 10],
    [5, 2, 9, 1, 5, 6],
    [12, 11, 13, 5, 6, 7],
    [38, 27, 43, 3, 9],
    [1],
    []
]

for i, test in enumerate(test_lists, 1):
    print(f"Original List {i}: {test}")
    print(f"Sorted List {i}: {merge_sort(test)}\n")

```

Output:

```

Original List 1: [38, 27, 43, 3, 9, 82, 10]
Sorted List 1:  [3, 9, 10, 27, 38, 43, 82]

Original List 2: [5, 2, 9, 1, 5, 6]
Sorted List 2:  [1, 2, 5, 5, 6, 9]

Sorted List 3:  [5, 6, 7, 11, 12, 13]

Original List 4: [38, 27, 43, 3, 9]
Sorted List 4:  [3, 9, 27, 38, 43]

Original List 5: [1]
Sorted List 5:  [1]

Original List 6: []
Sorted List 6:  []

```

Observation:

The program correctly implements the Merge Sort algorithm using divide and conquer. It sorts all given test lists properly, including single and empty lists. The time complexity is $O(n \log n)$ and the space complexity is $O(n)$.

Task:2

Prompt:

Generate a complete Python program that implements Binary Search.

Requirements:

- Create a function `binary_search(arr, target)`.
- Return the index if the target is found, otherwise return -1.
- Include a docstring explaining best, average, worst-case time complexity and space complexity.
- Add clear comments in the code.
- Include multiple test cases and print the results clearly.
- The script should run independently.

Code:

```
def binary_search(arr, target):
    """
    Performs binary search on a sorted list.

    Time Complexity:
    - Best Case: O(1) (when the target is at the middle)
    - Average Case: O(log n)
    - Worst Case: O(log n)

    Space Complexity:
    - O(1) for iterative version
    - O(log n) for recursive version due to call stack
    """

    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2 # Calculate the middle index

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    return -1 # Target was not found in the array
```

Test Cases

```
# ----- Test Cases -----
test_cases = [
    ([1, 2, 3, 4, 5], 3), # Target is in the middle
    ([1, 2, 3, 4, 5], 1), # Target is at the beginning
    ([1, 2, 3, 4, 5], 5), # Target is at the end
    ([1, 2, 3, 4, 5], 6), # Target is not in the list
    ([], 1)               # Empty list
]
for i, (arr, target) in enumerate(test_cases, 1):
    result = binary_search(arr, target)
    print(f"Test Case {i}: Array: {arr}, Target: {target} -> Result: {result}")
```

Output:

```
Test Case 1: Array: [1, 2, 3, 4, 5], Target: 3 -> Result: 2
Test Case 2: Array: [1, 2, 3, 4, 5], Target: 1 -> Result: 0
Test Case 3: Array: [1, 2, 3, 4, 5], Target: 5 -> Result: 4
Test Case 4: Array: [1, 2, 3, 4, 5], Target: 6 -> Result: -1
Test Case 5: Array: [], Target: 1 -> Result: -1
```

Observation:

The program correctly implements iterative Binary Search on a sorted list.
It handles different cases properly, including element found, not found, and empty list.
The time complexity is $O(\log n)$ and space complexity is $O(1)$.

Task 3:

Prompt:

Generate a Python program for a Smart Healthcare Appointment Scheduling System.

Requirements:

- Store appointment records (Appointment ID, Patient Name, Doctor Name, Appointment Time, Consultation Fee).
- Recommend and implement a suitable searching algorithm to search by Appointment ID.
- Recommend and implement suitable sorting algorithms to sort by Appointment Time and Consultation Fee.
- Justify the selected algorithms.
- Include docstrings, comments, and complexity explanations.
- Add sample data and demonstrate searching and sorting.
- The program should run independently.

Code:

```

class Appointment:
    """
    Represents a healthcare appointment.

    Attributes:
    - appointment_id (str): Unique identifier for the appointment.
    - patient_name (str): Name of the patient.
    - doctor_name (str): Name of the doctor.
    - appointment_time (str): Time of the appointment in 'HH:MM' format.
    - consultation_fee (float): Fee for the consultation.
    """

    def __init__(self, appointment_id, patient_name, doctor_name, appointment_time, consultation_fee):
        self.appointment_id = appointment_id
        self.patient_name = patient_name
        self.doctor_name = doctor_name
        self.appointment_time = appointment_time
        self.consultation_fee = consultation_fee

def binary_search_appointment(appointments, target_id):
    """
    Searches for an appointment by its ID using binary search.

    Time Complexity:
    -  $O(\log n)$  where  $n$  is the number of appointments.

    Space Complexity:
    -  $O(1)$  for iterative version.
    """
    left, right = 0, len(appointments) - 1

    while left <= right:
        mid = left + (right - left) // 2
        if appointments[mid].appointment_id == target_id:
            return appointments[mid]
        elif appointments[mid].appointment_id < target_id:
            left = mid + 1
        else:
            right = mid - 1

    return None # Appointment not found

def sort_appointments_by_time(appointments):
    """Sorts appointments by appointment time using Merge Sort.
    Time Complexity:  $O(n \log n)$ 
    Space Complexity:  $O(n)$ 
    """
    if len(appointments) <= 1:
        return appointments

    (function) def sort_appointments_by_time(appointments: Any) -> (Any | list)
    mid = len(appointments) // 2
    left = appointments[:mid]
    right = appointments[mid:]

    sort_appointments_by_time(left)
    sort_appointments_by_time(right)

    # Merge the sorted halves
    sorted_appointments = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].appointment_time < right[j].appointment_time:
            sorted_appointments.append(left[i])
            i += 1
        else:
            sorted_appointments.append(right[j])
            j += 1
    sorted_appointments.extend(left[i:])
    sorted_appointments.extend(right[j:])

    return sorted_appointments

```

```

def sort_appointments_by_time(appointments):

    return merge_by_time(left, right)
def merge_by_time(left, right):
    """Merges two lists of appointments sorted by time."""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].appointment_time <= right[j].appointment_time:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
def sort_appointments_by_fee(appointments):
    """Sorts appointments by consultation fee using Quick Sort.
    Time Complexity: O(n log n) on average, O(n^2) in worst case.
    Space Complexity: O(log n) on average, O(n) in worst case.
    """

    if len(appointments) <= 1:
        return appointments

# ----- Sample Data and Demonstration -----
appointments = [
    Appointment("A001", "John Doe", "Dr. Smith", "09:00", 100.0),
    Appointment("A002", "Jane Doe", "Dr. Brown", "10:30", 150.0),
    Appointment("A003", "Alice Johnson", "Dr. Smith", "11:00", 120.0),
    Appointment("A004", "Bob Lee", "Dr. Brown", "09:30", 80.0),
    Appointment("A005", "Charlie Kim", "Dr. Smith", "10:00", 110.0)
]
# Sort appointments by time
sorted_by_time = sort_appointments_by_time(appointments)
print("Appointments sorted by time:")
for appt in sorted_by_time:
    print(f"{appt.appointment_id}: {appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} - Fee: ${appt.consultation_fee}")
print("\n")
# Sort appointments by consultation fee
sorted_by_fee = sort_appointments_by_fee(appointments)
print("Appointments sorted by consultation fee:")
for appt in sorted_by_fee:
    print(f"{appt.appointment_id}: {appt.patient_name} with {appt.doctor_name} at {appt.appointment_time} - Fee: ${appt.consultation_fee}")
print("\n")
# Search for an appointment by ID
search_id = "A003"
found_appointment = binary_search_appointment(sorted_by_time, search_id)
if found_appointment:
    print(f"Appointment found: {found_appointment.appointment_id}: {found_appointment.patient_name} with {found_appointment.doctor_name} at {found_appointment.appointment_time} - Fee: ${found_appointment.consultation_fee}")
else:
    print(f"Appointment with ID {search_id} not found.")

```

Output:

```

Appointments sorted by time:
A001: John Doe with Dr. Smith at 09:00 - Fee: $100.0
A005: Charlie Kim with Dr. Smith at 10:00 - Fee: $110.0
A003: Alice Johnson with Dr. Smith at 11:00 - Fee: $120.0
A002: Jane Doe with Dr. Brown at 10:30 - Fee: $150.0

Appointment with ID A003 not found.

```

Justification:

Binary Search is used because it efficiently finds an appointment by ID in $O(\log n)$ time when the data is sorted.

Merge Sort is chosen for sorting by time because it guarantees $O(n \log n)$ performance and is stable.

Quick Sort is used for sorting by fee because it is fast on average with $O(n \log n)$ time complexity.

Task 4:

Prompt:

Generate a Python program for a Railway Ticket Reservation System.

Requirements:

- Store ticket details (Ticket ID, Passenger Name, Train Number, Seat Number, Travel Date).
- Implement an efficient search algorithm to find tickets by Ticket ID.
- Implement sorting algorithms to sort bookings by Travel Date and Seat Number.
- Justify the chosen algorithms.
- Include docstrings, comments, and complexity explanation.
- Add sample data and demonstrate searching and sorting.
- The program should run independently.

Code:

```
class Ticket:
```

```
    """
```

```
    Represents a railway ticket reservation.
```

Attributes:

- ticket_id (str): Unique identifier for the ticket.
- passenger_name (str): Name of the passenger.
- train_number (str): Number of the train.

- seat_number (str): Seat number assigned to the passenger.

- travel_date (str): Date of travel in 'YYYY-MM-DD' format.

"""

```
def __init__(self, ticket_id, passenger_name, train_number, seat_number, travel_date):
```

```
    self.ticket_id = ticket_id
```

```
    self.passenger_name = passenger_name
```

```
    self.train_number = train_number
```

```
    self.seat_number = seat_number
```

```
    self.travel_date = travel_date
```

```
def binary_search_ticket(tickets, target_id):
```

"""

Searches for a ticket by its ID using binary search.

Time Complexity:

- $O(\log n)$ where n is the number of tickets.

Space Complexity:

- $O(1)$ for iterative version.

"""

```
left, right = 0, len(tickets) - 1
```

```
while left <= right:
```

```
    mid = left + (right - left) // 2
```

```
    if tickets[mid].ticket_id == target_id:
```

```
        return tickets[mid]
```

```
    elif tickets[mid].ticket_id < target_id:
```

```
        left = mid + 1
```

```
    else:
```

```
        right = mid - 1
```

```

    return None # Ticket not found
def sort_tickets_by_date(tickets):
    """Sorts tickets by travel date using Merge Sort.
    Time Complexity: O(n log n)
    Space Complexity: O(n)
    """
    if len(tickets) <= 1:
        return tickets

    mid = len(tickets) // 2
    left = sort_tickets_by_date(tickets[:mid])
    right = sort_tickets_by_date(tickets[mid:])

    return merge_by_date(left, right)
def merge_by_date(left, right):
    """Merges two lists of tickets sorted by travel date."""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].travel_date <= right[j].travel_date:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

```

```

def sort_tickets_by_seat(tickets):
    """Sorts tickets by seat number using Quick Sort.

    Time Complexity:  $O(n \log n)$  on average,  $O(n^2)$  in worst case.
    Space Complexity:  $O(\log n)$  on average,  $O(n)$  in worst case.
    """

    if len(tickets) <= 1:
        return tickets

    pivot = tickets[len(tickets) // 2].seat_number
    left = [x for x in tickets if x.seat_number < pivot]
    middle = [x for x in tickets if x.seat_number == pivot]
    right = [x for x in tickets if x.seat_number > pivot]

    return sort_tickets_by_seat(left) + middle + sort_tickets_by_seat(right)

# ----- Sample Data and Demonstration -----
tickets = [
    Ticket("T001", "Alice Smith", "Train 101", "A1", "2024-07-01"),
    Ticket("T002", "Bob Johnson", "Train 102", "B2", "2024-07-02"),
    Ticket("T003", "Charlie Brown", "Train 101", "A2", "2024-07-01"),
    Ticket("T004", "David Lee", "Train 103", "C1", "2024-07-03"),
    Ticket("T005", "Eve Davis", "Train 102", "B1", "2024-07-02")
]

# Sort tickets by travel date
sorted_by_date = sort_tickets_by_date(tickets)
print("Tickets sorted by travel date:")

for ticket in sorted_by_date:
    print(f"{ticket.ticket_id}: {ticket.passenger_name} on {ticket.train_number} seat {ticket.seat_number} traveling on {ticket.travel_date}")

print("\n")

# Sort tickets by seat number
sorted_by_seat = sort_tickets_by_seat(tickets)

```

```

print("Tickets sorted by seat number:")

for ticket in sorted_by_seat:

    print(f"{ticket.ticket_id}: {ticket.passenger_name} on {ticket.train_number} seat  

{ticket.seat_number} traveling on {ticket.travel_date}")

print("\n")

# Search for a ticket by ID

search_id = "T003"

found_ticket = binary_search_ticket(sorted_by_date, search_id)

if found_ticket:

    print(f"Ticket found: {found_ticket.ticket_id}: {found_ticket.passenger_name} on  

{found_ticket.train_number} seat {found_ticket.seat_number} traveling on  

{found_ticket.travel_date}")

else: print(f"Ticket with ID {search_id} not found.")

output:

```

```

Tickets sorted by travel date:
T001: Alice Smith on Train 101 seat A1 traveling on 2024-07-01
T003: Charlie Brown on Train 101 seat A2 traveling on 2024-07-01
T002: Bob Johnson on Train 102 seat B2 traveling on 2024-07-02
T005: Eve Davis on Train 102 seat B1 traveling on 2024-07-02
T004: David Lee on Train 103 seat C1 traveling on 2024-07-03

Tickets sorted by seat number:
T001: Alice Smith on Train 101 seat A1 traveling on 2024-07-01
T003: Charlie Brown on Train 101 seat A2 traveling on 2024-07-01
T005: Eve Davis on Train 102 seat B1 traveling on 2024-07-02
T002: Bob Johnson on Train 102 seat B2 traveling on 2024-07-02
T004: David Lee on Train 103 seat C1 traveling on 2024-07-03

Ticket with ID T003 not found.

```

Justification:

Binary Search is used to search ticket by ID because it works in $O(\log n)$ time when the list is sorted, making it much faster than linear search for large data.

Merge Sort is used for sorting by travel date because it guarantees $O(n \log n)$ time complexity and maintains stable ordering.

Quick Sort is used for sorting by seat number because it is efficient on average with $O(n \log n)$ performance and is simple to implement.

Task 5:

Prompt:

Generate a Python program for a Smart Hostel Room Allocation System.

Requirements:

- Store student details (Student ID, Room Number, Floor, Allocation Date).
- Implement an optimized search algorithm to find records by Student ID.
- Implement sorting algorithms to sort by Room Number and Allocation Date.
- Justify the chosen algorithms.
- Include docstrings, comments, and complexity explanation.
- Add sample data and demonstrate searching and sorting.
- The program should run independently.

Code:

```
def binary_search(records, student_id):
    """
    Searches for a student allocation record using Binary Search.

    Algorithm: Binary Search (requires records sorted by Student ID)

    Time Complexity:
    - Best Case:  $O(1)$ 
    - Average Case:  $O(\log n)$ 
    - Worst Case:  $O(\log n)$ 

    Space Complexity:
    -  $O(1)$ 
    """

    low = 0
    high = len(records) - 1

    while low <= high:
        mid = (low + high) // 2

        if records[mid]["student_id"] == student_id:
            return mid
        elif records[mid]["student_id"] < student_id:
            low = mid + 1
        else:
            high = mid - 1
```

```

records = [
    {"student_id": 101, "room_number": 12, "floor": 1, "allocation_date": "2025-01-10"},
    {"student_id": 105, "room_number": 8, "floor": 1, "allocation_date": "2025-01-05"},
    {"student_id": 102, "room_number": 15, "floor": 2, "allocation_date": "2025-01-12"},
    {"student_id": 104, "room_number": 5, "floor": 1, "allocation_date": "2025-01-02"},
    {"student_id": 103, "room_number": 20, "floor": 3, "allocation_date": "2025-01-15"},
]

# Binary search requires sorted data by student_id
records_sorted_by_id = sorted(records, key=lambda x: x["student_id"])

# ----- Demonstration -----

print("Original Records:")
for r in records:
    print(r)

# Search Example
search_id = 102
index = binary_search(records_sorted_by_id, search_id)

print("\nSearching for Student ID:", search_id)
if index != -1:
    print("Record Found:", records_sorted_by_id[index])
else:
    print("Record Not Found")

# Sort by Room Number

```

0

```

        return -1

def sort_by_room(records):
    """
    Sorts records by Room Number using Python's built-in Timsort.

    Time Complexity: O(n log n)
    Space Complexity: O(n)
    """
    return sorted(records, key=lambda x: x["room_number"])

def sort_by_date(records):
    """
    Sorts records by Allocation Date.

    Time Complexity: O(n log n)
    Space Complexity: O(n)
    """
    return sorted(records, key=lambda x: x["allocation_date"])

# ----- Sample Data -----

```

Output:

```

Original Records:
{'student_id': 101, 'room_number': 12, 'floor': 1, 'allocation_date': '2025-01-10'}
{'student_id': 105, 'room_number': 8, 'floor': 1, 'allocation_date': '2025-01-05'}
{'student_id': 102, 'room_number': 15, 'floor': 2, 'allocation_date': '2025-01-12'}
{'student_id': 104, 'room_number': 5, 'floor': 1, 'allocation_date': '2025-01-02'}
{'student_id': 103, 'room_number': 20, 'floor': 3, 'allocation_date': '2025-01-15'}

Searching for Student ID: 102
Record Found: {'student_id': 102, 'room_number': 15, 'floor': 2, 'allocation_date': '2025-01-12'}

Sorted by Room Number:
Sorted by Allocation Date:
{'student_id': 104, 'room_number': 5, 'floor': 1, 'allocation_date': '2025-01-02'}
{'student_id': 105, 'room_number': 8, 'floor': 1, 'allocation_date': '2025-01-05'}
{'student_id': 101, 'room_number': 12, 'floor': 1, 'allocation_date': '2025-01-10'}
{'student_id': 102, 'room_number': 15, 'floor': 2, 'allocation_date': '2025-01-12'}
{'student_id': 103, 'room_number': 20, 'floor': 3, 'allocation_date': '2025-01-15'}

```

Justification:

Binary Search is used because the records are sorted by student_id, allowing fast searching in **O(log n)** time instead of O(n) linear search. It is efficient for large datasets and uses **O(1)** space.

Timsort (Python's built-in sorted()) is used for sorting by room number and allocation date because it guarantees **$O(n \log n)$** time complexity, is stable, and is highly optimized for real-world data.

Thus, the chosen algorithms ensure **fast searching and efficient, reliable sorting**.

Task:6

Prompt:

Generate a Python program for an Online Movie Streaming Platform.

Requirements:

- Store movie details (Movie ID, Title, Genre, Rating, Release Year).
- Implement an efficient search algorithm to find movies by Movie ID.
- Implement sorting algorithms to sort by Rating and Release Year.
- Justify the chosen algorithms.
- Include docstrings, comments, and complexity explanation.
- Add sample data and demonstrate searching and sorting.
- The program should run independently.

Code:

class Movie:

"""

Represents a movie in the streaming platform.

Attributes:

- movie_id (str): Unique identifier for the movie.
- title (str): Title of the movie.
- genre (str): Genre of the movie.
- rating (float): Rating of the movie (0.0 to 10.0).
- release_year (int): Year the movie was released.

"""

def __init__(self, movie_id, title, genre, rating, release_year):

self.movie_id = movie_id

self.title = title

self.genre = genre

```
        self.rating = rating

        self.release_year = release_year

def binary_search_movie(movies, target_id):
    """

    Searches for a movie by its ID using binary search.
```

Time Complexity:

- $O(\log n)$ where n is the number of movies.

Space Complexity:

- $O(1)$ for iterative version.

```
    """

    left, right = 0, len(movies) - 1

    while left <= right:

        mid = left + (right - left) // 2

        if movies[mid].movie_id == target_id:

            return movies[mid]

        elif movies[mid].movie_id < target_id:

            left = mid + 1

        else:

            right = mid - 1

    return None # Movie not found
```

```
def sort_movies_by_rating(movies):
    """Sorts movies by rating using Merge Sort.

    Time Complexity:  $O(n \log n)$ 

    Space Complexity:  $O(n)$ 

    """
```

```
    if len(movies) <= 1:
```

```

    return movies

mid = len(movies) // 2
left = sort_movies_by_rating(movies[:mid])
right = sort_movies_by_rating(movies[mid:])

return merge_by_rating(left, right)
def merge_by_rating(left, right):
    """Merges two lists of movies sorted by rating."""
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i].rating >= right[j].rating: # Higher rating first
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
def sort_movies_by_release_year(movies):
    """Sorts movies by release year using Quick Sort.

    Time Complexity:  $O(n \log n)$  on average,  $O(n^2)$  in worst case.
    Space Complexity:  $O(\log n)$  on average,  $O(n)$  in worst case.
    """
    if len(movies) <= 1:
        return movies

```

```

pivot = movies[len(movies) // 2].release_year

left = [x for x in movies if x.release_year < pivot]

middle = [x for x in movies if x.release_year == pivot]

right = [x for x in movies if x.release_year > pivot]


return sort_movies_by_release_year(left) + middle + sort_movies_by_release_year(right)

# ----- Sample Data and Demonstration -----

movies = [

    Movie("M001", "Inception", "Sci-Fi", 8.8, 2010),

    Movie("M002", "The Godfather", "Crime", 9.2, 1972),

    Movie("M003", "The Dark Knight", "Action", 9.0, 2008),

    Movie("M004", "Pulp Fiction", "Crime", 8.9, 1994),

    Movie("M005", "The Shawshank Redemption", "Drama", 9.3, 1994)

]

# Sort movies by rating

sorted_by_rating = sort_movies_by_rating(movies)

print("Movies sorted by rating:")

for movie in sorted_by_rating:

    print(f"{movie.movie_id}: {movie.title} - Genre: {movie.genre}, Rating: {movie.rating}, Release Year: {movie.release_year}")

print("\n")

# Sort movies by release year

sorted_by_release_year = sort_movies_by_release_year(movies)

print("Movies sorted by release year:")

for movie in sorted_by_release_year:

    print(f"{movie.movie_id}: {movie.title} - Genre: {movie.genre}, Rating: {movie.rating}, Release Year: {movie.release_year}")

print("\n")

# Search for a movie by ID

search_id = "M003"

found_movie = binary_search_movie(sorted_by_rating, search_id)

if found_movie:

```

```
print(f"Movie found: {found_movie.movie_id}: {found_movie.title} - Genre: {found_movie.genre},  
Rating: {found_movie.rating}, Release Year: {found_movie.release_year}")
```

```
else: print(f"Movie with ID {search_id} not found.")
```

Output:

```
Movies sorted by rating:  
M005: The Shawshank Redemption - Genre: Drama, Rating: 9.3, Release Year: 1994  
M002: The Godfather - Genre: Crime, Rating: 9.2, Release Year: 1972  
M003: The Dark Knight - Genre: Action, Rating: 9.0, Release Year: 2008  
M004: Pulp Fiction - Genre: Crime, Rating: 8.9, Release Year: 1994  
M001: Inception - Genre: Sci-Fi, Rating: 8.8, Release Year: 2010  
M005: The Shawshank Redemption - Genre: Drama, Rating: 9.3, Release Year: 1994  
M003: The Dark Knight - Genre: Action, Rating: 9.0, Release Year: 2008  
M001: Inception - Genre: Sci-Fi, Rating: 8.8, Release Year: 2010  
  
Movie found: M003: The Dark Knight - Genre: Action, Rating: 9.0, Release Year: 2008
```

Justification:

Binary Search is used to find a movie by movie_id because it works in $O(\log n)$ time when the list is sorted, making it efficient for large movie collections.

Merge Sort is used for sorting by rating because it guarantees $O(n \log n)$ time complexity and provides stable, consistent performance.

Quick Sort is used for sorting by release year because it is fast on average ($O(n \log n)$) and simple to implement for numerical data.

Task 7:

Prompt:

Generate a Python program for a Smart Agriculture Crop Monitoring System.

Requirements:

- Store crop details (Crop ID, Crop Name, Soil Moisture Level, Temperature, Yield Estimate).
- Implement an efficient search algorithm to find crops by Crop ID.
- Implement sorting algorithms to sort by Soil Moisture Level and Yield Estimate.
- Justify the chosen algorithms.
- Include docstrings, comments, and complexity explanation.
- Add sample data and demonstrate searching and sorting.
- The program should run independently.

Code:

```
def binary_search(crops, crop_id):
```

```
"""
```

Searches for a crop by Crop ID using Binary Search.

Time Complexity:

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$

Space Complexity:

- $O(1)$

```
"""
```

```
low = 0
```

```
high = len(crops) - 1
```

```
while low <= high:
```

```
    mid = (low + high) // 2
```

```
    if crops[mid]["crop_id"] == crop_id:
```

```
        return mid
```

```
    elif crops[mid]["crop_id"] < crop_id:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

```
return -1
```

```
def sort_by_moisture(crops):
```

```
    """
```

Sort crops by Soil Moisture Level (ascending).

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

"""

return sorted(crops, key=lambda x: x["moisture"])

def sort_by_yield(crops):

"""

Sort crops by Yield Estimate (ascending).

Time Complexity: $O(n \log n)$

Space Complexity: $O(n)$

"""

return sorted(crops, key=lambda x: x["yield_estimate"])

----- Sample Data -----

crops = [

{"crop_id": 101, "crop_name": "Wheat", "moisture": 30, "temperature": 25, "yield_estimate": 200},

{"crop_id": 105, "crop_name": "Rice", "moisture": 45, "temperature": 28, "yield_estimate": 300},

{"crop_id": 102, "crop_name": "Corn", "moisture": 35, "temperature": 27, "yield_estimate": 250},

{"crop_id": 104, "crop_name": "Barley", "moisture": 25, "temperature": 22, "yield_estimate": 180},

{"crop_id": 103, "crop_name": "Soybean", "moisture": 40, "temperature": 26, "yield_estimate": 270},

]

Binary Search requires sorted list by crop_id

crops_sorted_by_id = sorted(crops, key=lambda x: x["crop_id"])

----- Demonstration -----

print("Original Crop Records:")

```
for crop in crops:
```

```
    print(crop)
```

```
# Search Example
```

```
search_id = 102
```

```
index = binary_search(crops_sorted_by_id, search_id)
```

```
print("\nSearching for Crop ID:", search_id)
```

```
if index != -1:
```

```
    print("Crop Found:", crops_sorted_by_id[index])
```

```
else:
```

```
    print("Crop Not Found")
```

```
# Sort by Moisture
```

```
print("\nSorted by Soil Moisture Level:")
```

```
for crop in sort_by_moisture(crops):
```

```
    print(crop)
```

```
# Sort by Yield Estimate
```

```
print("\nSorted by Yield Estimate:")
```

```
for crop in sort_by_yield(crops):
```

```
    print(crop)
```

```
Output:
```



```

Original Crop Records:
{'crop_id': 101, 'crop_name': 'Wheat', 'moisture': 30, 'temperature': 25, 'yield_estimate': 200}
{'crop_id': 105, 'crop_name': 'Rice', 'moisture': 45, 'temperature': 28, 'yield_estimate': 300}
{'crop_id': 102, 'crop_name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield_estimate': 250}
{'crop_id': 104, 'crop_name': 'Barley', 'moisture': 25, 'temperature': 22, 'yield_estimate': 180}
{'crop_id': 103, 'crop_name': 'Soybean', 'moisture': 40, 'temperature': 26, 'yield_estimate': 270}

Searching for Crop ID: 102
Crop Found: {'crop_id': 102, 'crop_name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield_estimate': 250}

Sorted by Soil Moisture Level:
{'crop_id': 104, 'crop_name': 'Barley', 'moisture': 25, 'temperature': 22, 'yield_estimate': 180}
{'crop_id': 101, 'crop_name': 'Wheat', 'moisture': 30, 'temperature': 25, 'yield_estimate': 200}
{'crop_id': 102, 'crop_name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield_estimate': 250}
{'crop_id': 103, 'crop_name': 'Soybean', 'moisture': 40, 'temperature': 26, 'yield_estimate': 270}
{'crop_id': 105, 'crop_name': 'Rice', 'moisture': 45, 'temperature': 28, 'yield_estimate': 300}

Sorted by Yield Estimate:
{'crop_id': 104, 'crop_name': 'Barley', 'moisture': 25, 'temperature': 22, 'yield_estimate': 180}
{'crop_id': 101, 'crop_name': 'Wheat', 'moisture': 30, 'temperature': 25, 'yield_estimate': 200}
{'crop_id': 102, 'crop_name': 'Corn', 'moisture': 35, 'temperature': 27, 'yield_estimate': 250}
{'crop_id': 103, 'crop_name': 'Soybean', 'moisture': 40, 'temperature': 26, 'yield_estimate': 270}
{'crop_id': 105, 'crop_name': 'Rice', 'moisture': 45, 'temperature': 28, 'yield_estimate': 300}

```

Justification:

Binary Search is used to find a crop by crop_id because the list is sorted by ID, allowing efficient searching in **$O(\log n)$** time instead of $O(n)$. It also uses **$O(1)$** space.

Python's built-in sorted() (Timsort) is used for sorting by moisture and yield because it guarantees **$O(n \log n)$** time complexity, is stable, and is highly optimized for real-world data.

Thus, the algorithms ensure **fast searching and efficient, reliable sorting**.

Task 8:

Prompt:

Generate a Python program for an Airport Flight Management System.

Requirements:

- Store flight details (Flight ID, Airline Name, Departure Time, Arrival Time, Status).
- Implement an efficient search algorithm to find flights by Flight ID.
- Implement sorting algorithms to sort by Departure Time and Arrival Time.
- Justify the chosen algorithms.
- Include docstrings, comments, and complexity explanation.
- Add sample data and demonstrate searching and sorting.
- The program should run independently.

Code:

```
class Flight:
```

```
    """
```

```
    Represents a flight in the airport management system.
```

Attributes:

- flight_id (str): Unique identifier for the flight.
- airline_name (str): Name of the airline.
- departure_time (str): Departure time in 'HH:MM' format.
- arrival_time (str): Arrival time in 'HH:MM' format.
- status (str): Status of the flight (e.g., "On Time", "Delayed", "Cancelled").

"""

```
def __init__(self, flight_id, airline_name, departure_time, arrival_time, status):
```

```
    self.flight_id = flight_id
```

```
    self.airline_name = airline_name
```

```
    self.departure_time = departure_time
```

```
    self.arrival_time = arrival_time
```

```
    self.status = status
```

```
def binary_search_flight(flights, target_id):
```

```
    """
```

Searches for a flight by its ID using binary search.

Time Complexity:

- $O(\log n)$ where n is the number of flights.

Space Complexity:

- $O(1)$ for iterative version.

```
    """
```

```
left, right = 0, len(flights) - 1
```

```
while left <= right:
```

```
    mid = left + (right - left) // 2
```

```
    if flights[mid].flight_id == target_id:
```

```

        return flights[mid]

    elif flights[mid].flight_id < target_id:

        left = mid + 1

    else:

        right = mid - 1

    return None # Flight not found

def sort_flights_by_departure(flights):

    """Sorts flights by departure time using Merge Sort.

    Time Complexity: O(n log n)

    Space Complexity: O(n)

    """

    if len(flights) <= 1:

        return flights

    mid = len(flights) // 2

    left = sort_flights_by_departure(flights[:mid])

    right = sort_flights_by_departure(flights[mid:])

    return merge_by_departure(left, right)

def merge_by_departure(left, right):

    """Merges two lists of flights sorted by departure time."""

    result = []

    i = j = 0

    while i < len(left) and j < len(right):

        if left[i].departure_time <= right[j].departure_time:

            result.append(left[i])

            i += 1

        else:

            result.append(right[j])

```

```
j += 1
```

```
result.extend(left[i:])
```

```
result.extend(right[j:])
```

```
return result
```

```
def sort_flights_by_arrival(flights):
```

```
    """Sorts flights by arrival time using Quick Sort.
```

```
    Time Complexity:  $O(n \log n)$  on average,  $O(n^2)$  in worst case.
```

```
    Space Complexity:  $O(\log n)$  on average,  $O(n)$  in worst case.
```

```
    """
```

```
    if len(flights) <= 1:
```

```
        return flights
```

```
    pivot = flights[len(flights) // 2].arrival_time
```

```
    left = [x for x in flights if x.arrival_time < pivot]
```

```
    middle = [x for x in flights if x.arrival_time == pivot]
```

```
    right = [x for x in flights if x.arrival_time > pivot]
```

```
    return sort_flights_by_arrival(left) + middle + sort_flights_by_arrival(right)
```

```
# ----- Sample Data and Demonstration -----
```

```
flights = [
```

```
    Flight("F001", "Airline A", "08:00", "10:00", "On Time"),
```

```
    Flight("F002", "Airline B", "09:30", "11:30", "Delayed"),
```

```
    Flight("F003", "Airline C", "07:45", "09:45", "Cancelled"),
```

```
    Flight("F004", "Airline D", "10:15", "12:15", "On Time"),
```

```
    Flight("F005", "Airline E", "06:30", "08:30", "On Time")
```

```
]
```

```
# Sort flights by departure time
```

```
sorted_by_departure = sort_flights_by_departure(flights)
```

```
print("Flights sorted by departure time:")
```

```
for flight in sorted_by_departure:
```

```

    print(f"{flight.flight_id}: {flight.airline_name} - Departure: {flight.departure_time}, Arrival:
{flight.arrival_time}, Status: {flight.status}")

print("\n")

# Sort flights by arrival time

sorted_by_arrival = sort_flights_by_arrival(flights)

print("Flights sorted by arrival time:")

for flight in sorted_by_arrival:

    print(f"{flight.flight_id}: {flight.airline_name} - Departure: {flight.departure_time}, Arrival:
{flight.arrival_time}, Status: {flight.status}")

print("\n")

# Search for a flight by ID

search_id = "F003"

found_flight = binary_search_flight(sorted_by_departure, search_id)

if found_flight:

    print(f"Flight found: {found_flight.flight_id}: {found_flight.airline_name} - Departure:
{found_flight.departure_time}, Arrival: {found_flight.arrival_time}, Status: {found_flight.status}")

else: print(f"Flight with ID {search_id} not found.")

```

Output:

```

Flights sorted by departure time:
F005: Airline E - Departure: 06:30, Arrival: 08:30, Status: On Time
F003: Airline C - Departure: 07:45, Arrival: 09:45, Status: Cancelled
F001: Airline A - Departure: 08:00, Arrival: 10:00, Status: On Time
F002: Airline B - Departure: 09:30, Arrival: 11:30, Status: Delayed
F003: Airline C - Departure: 07:45, Arrival: 09:45, Status: Cancelled
F001: Airline A - Departure: 08:00, Arrival: 10:00, Status: On Time
F002: Airline B - Departure: 09:30, Arrival: 11:30, Status: Delayed
F004: Airline D - Departure: 10:15, Arrival: 12:15, Status: On Time

F003: Airline C - Departure: 07:45, Arrival: 09:45, Status: Cancelled
F001: Airline A - Departure: 08:00, Arrival: 10:00, Status: On Time
F002: Airline B - Departure: 09:30, Arrival: 11:30, Status: Delayed
F004: Airline D - Departure: 10:15, Arrival: 12:15, Status: On Time

Flight with ID F003 not found.

```

Justification:

Binary Search is used to search for a flight by flight_id because it reduces the search space by half each step, giving $O(\log n)$ time complexity. It is much faster than linear search for large datasets.

Merge Sort is used for sorting by departure time because it guarantees $O(n \log n)$ performance and provides stable sorting.

Quick Sort is used for sorting by arrival time because it is efficient on average ($O(n \log n)$) and works well for time-based comparisons.